

6.5) Mapping Cache Memory

• Cache Initialization

1. One more aspect of cache organization that must be taken into consideration is the problem of initialization.
2. The cache is initialized when power is applied to the computer or when the main memory is loaded with a complete set of programs from auxiliary memory.
3. After initialization the cache is considered to be empty, but in effect it contains some nonvalid data.
4. It is customary to include with each word in cache a valid bit to indicate whether or not the word contains valid data.
5. The cache is initialized by clearing all the valid bits to 0.
6. The valid bit of a particular cache word is set to 1 the first time this word is loaded from main memory and stays set unless the cache has to be initialized again.
7. The introduction of the valid bit means that a word in cache is not replaced by another word unless the valid bit is set to 1 and a mismatch of tags occurs.
8. If the valid bit happens to be 0, the new word automatically replaces the invalid data.
9. Thus the initialization condition has the effect of forcing misses from the cache until it fills with valid data.

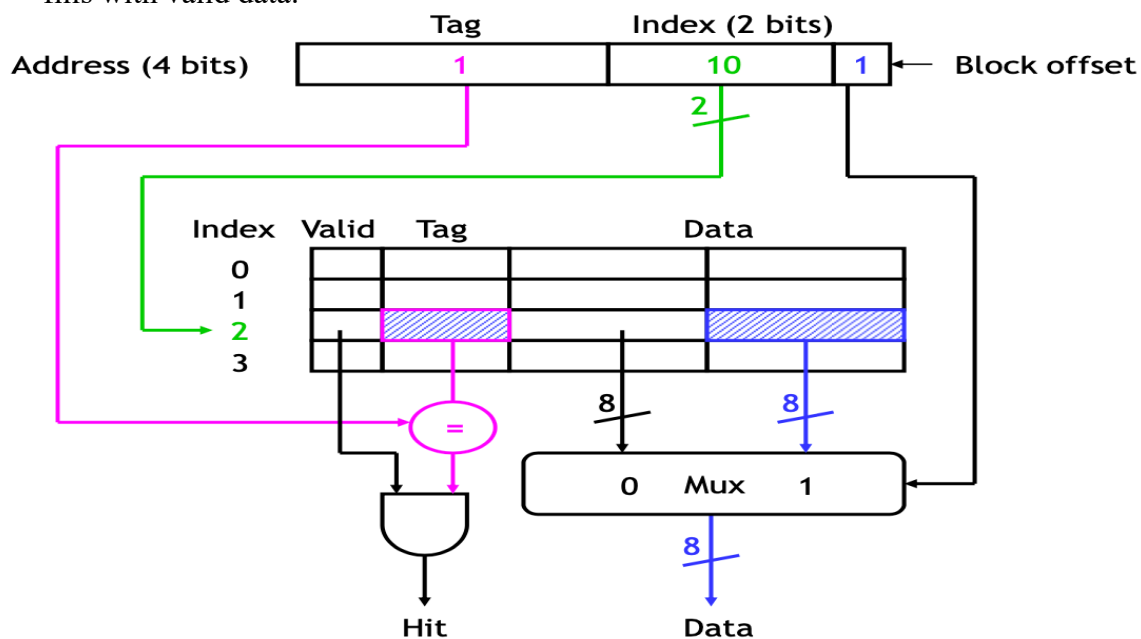


Fig: Illustration of valid bit in cache initialization

• Mapping Cache Memory functions:

Used to map a particular block of main memory to a particular block of main memory to a particular block of cache. The mapping function is used to transfer the block from main memory to cache memory. Three mapping function exists.

i.) Direct mapping

ii.) Associative mapping

iii.) Set Associative mapping i.) Direct Mapping:

- Address length = $(s + w)$ bits
- Number of addressable units = $2^{(s+w)}$ words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $2^{(s+w)}/2^w = 2^s$
- Number of lines (or index) in cache = $m = 2^r$
- Size of tag = $(s-r)$ bits

In direct mapping, block K of main memory maps into block k modulo m ($K \% m$) of the cache. Here m is the total no. of blocks in cache. Here value of m is 128.

Direct mapping pros n cons:

- Simple
- Inexpensive
- Fixed location for given block

— If a program accesses 2 blocks that map to the same line repeatedly, cache misses are very high

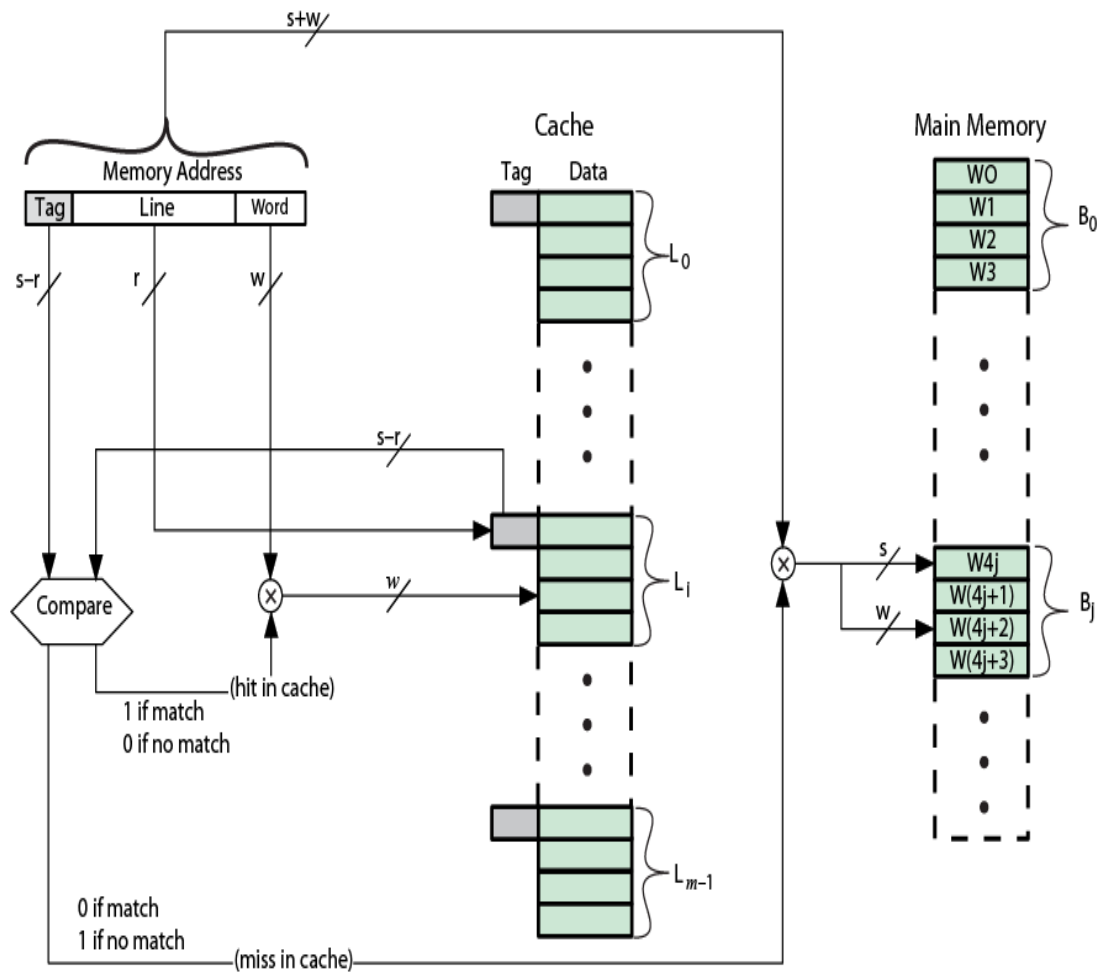


Figure: Direct mapping.

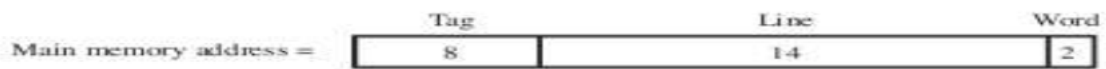


Fig: Address format

Ii)Associative Mapping:

- Address length = $(s + w)$ bits
- Number of addressable units = $2^{(s+w)}$ words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $2^{(s+w)}/2^w = 2^s$
- Number of lines in cache = undetermined
- Size of tag=sbits

In this mapping technique, any block of main memory can potentially reside in any cache block position. It is much more flexible mapping technique.

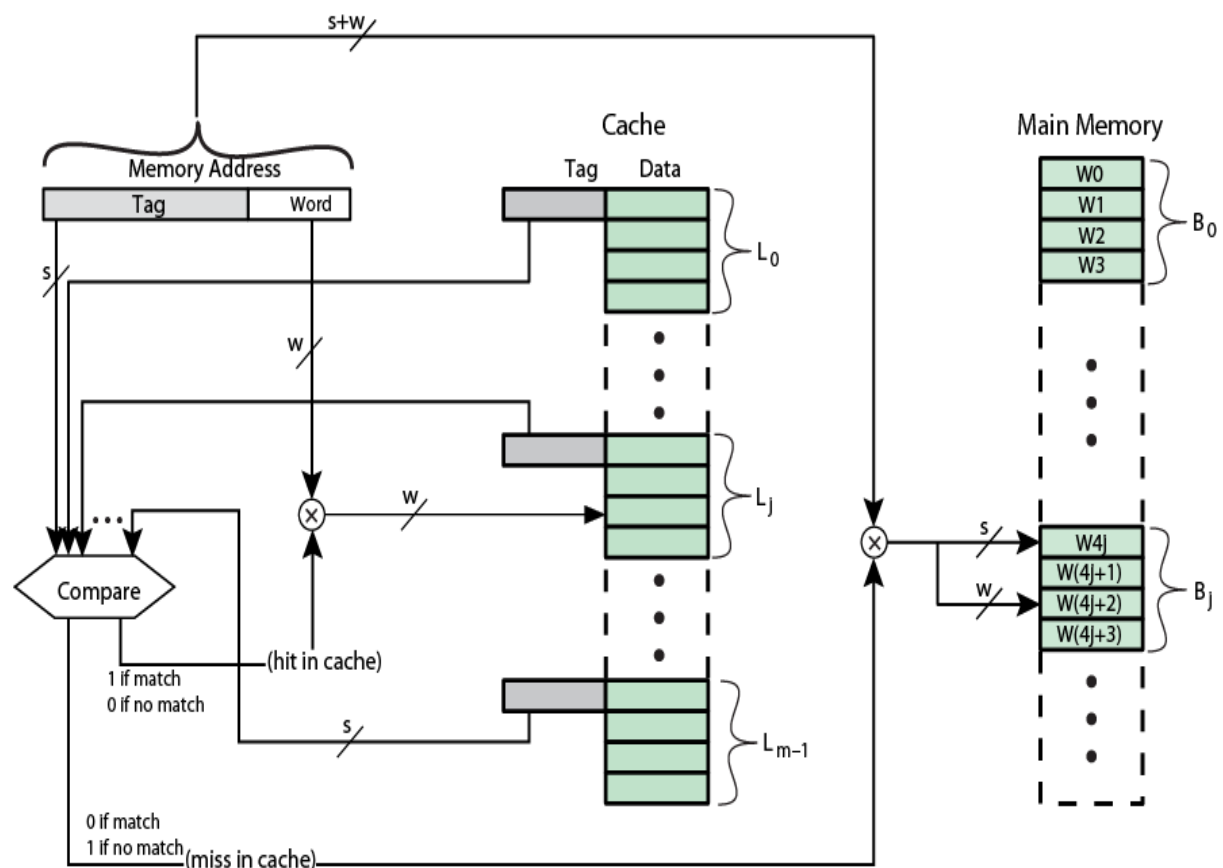


Fig: Associative mapping



Fig: Address format

iii.) Set Associative mapping:

- Address length = $(s + w)$ bits
- Number of addressable units = $2^{(s+w)}$ words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = 2^w
- Number of lines in set = k
- Number of sets = $v = 2^d$
- Number of lines in cache = $kv = k \cdot 2^d$
- Size of tag = $(s - d)$ bits

In this method blocks of cache are grouped into sets, & the mapping allows a block of main memory to reside in any block of a specific set. From the flexibility point of view it is between the other two method.

This reduced searching overhead, because the search is restricted to no. of sets, instead of no. of block.

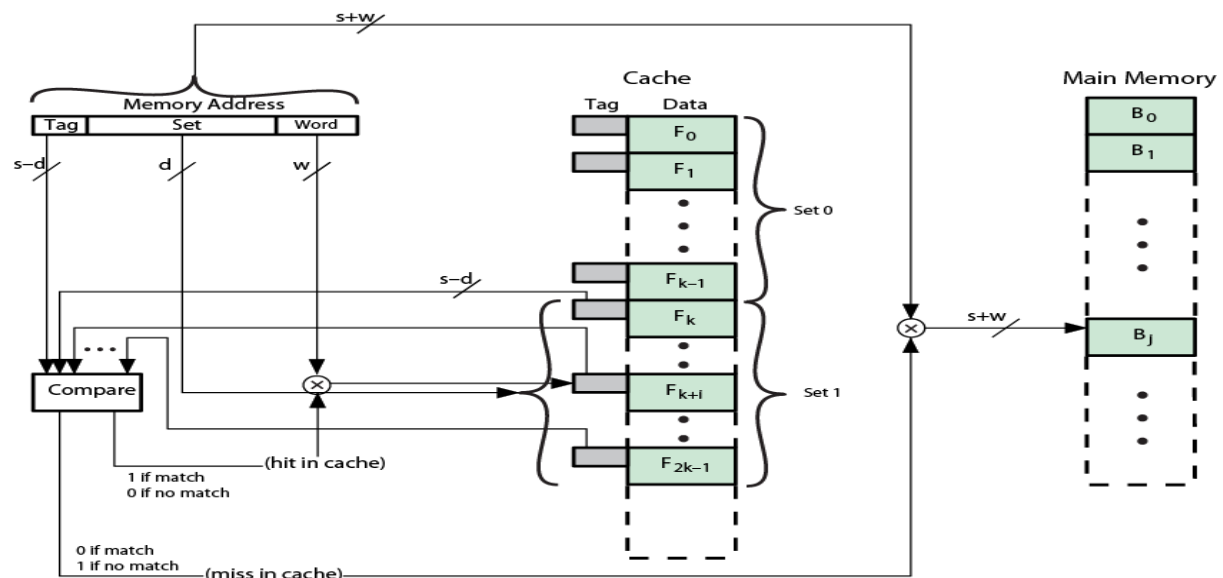


Fig: Set associative mapping



Fig: Address format

- **Write policy:**

The system must not overwrite a cache block unless main memory is up to date.

Multiple CPUs may have individual caches, so there will be problem of coherence .

I/O may address main memory directly and may bring inconsistency in data if the addressed location is cached by some other CPUs.

- a) Write through**

All writes go to main memory as well as cache

Multiple CPUs can monitor main memory traffic to keep local (to CPU) cache up to date

Lots of traffic

Slows down writes

- b) Write back**

Updates initially made in cache only

Update bit for cache slot is set when update occurs

If block is to be replaced, write to main memory only if update bit is set

Other caches get out of sync

I/O must access main memory through cache

N.B. 15% of memory references are writes

- **Replacement algorithm:**

Direct mapping

- No choice
 - Each block only maps to one line
 - Replace that line

Replacement Algorithms Associative & Set Associative

- Hardware implemented algorithm (speed)
- Least Recently used (LRU)
 - e.g. in 2 ways set associative
 - Which of the 2 block is lru?
- First in first out (FIFO)
 - replace block that has been in cache longest
- Least frequently used
 - replace block which has had fewest hits
- Random